PREGUNTA 1

XML

He dissenyat el document amb un únic element arrel <factura>, tal com exigeix un XML ben format, i l'he dividit en blocs (capçalera, emissor, client, línies i totals) que reprodueixen les seccions de la factura en paper, de manera que és fàcil de llegir i de validar. El número de factura i el de client són atributs perquè identifiquen l'entitat, mentre que la resta de dades són elements, ja que són contingut i permeten subestructures com l'adreça, que desglosso en carrer, codi postal i població per poder filtrar per cada camp. Les línies de la factura es repeteixen com a elements <linia> dins de <linies>, així es poden afegir productes sense canviar l'estructura. La data va en format ISO 8601 per evitar ambigüitats, els imports duen punt decimal i sense el símbol €, que s'indica a l'atribut moneda per poder operar amb els valors, i el telèfon i el codi postal es guarden com a text per no perdre el zero inicial (07001). Finalment, declar la codificació UTF-8 perquè els accents i la ç es vegin bé, i l'XML permetria validar-lo després amb un XSD o DTD.

JSON

He seguit la mateixa organització lògica, amb un objecte principal factura que conté objectes niats per a l'emissor, el client, l'adreça i els totals, de manera que cada bloc queda agrupat i el seu significat és clar. Les línies de factura es representen amb un array d'objectes, que és l'estructura natural de JSON per a llistes d'elements repetits i permet afegir o llevar línies lliurement. Aprofit els tipus natius de JSON: quantitats, preus, IVA i imports van com a nombres, cosa que permet fer càlculs directament, mentre que el telèfon, el CIF i el codi postal van com a cadenes de text perquè són identificadors i no s'operen. Mantinc els mateixos criteris que a l'XML (data en format ISO, decimals amb punt, moneda separada

del valor, claus sense espais ni caràcters especials), de manera que ambdós documents són equivalents. JSON resulta més compacte i lleuger que XML, encara que no admet atributs ni comentaris, i és el format més habitual per intercanviar dades en APIs.

PREGUNTA 2

XML

He dissenyat el document amb un únic element arrel <correu>, com exigeix un XML ben format, i l'he dividit en tres blocs (capçalera, cos i adjunts) que corresponen a les parts d'un correu electrònic de la imatge. A la capçalera hi van l'assumpte, el remitent i els destinataris; el remitent és un element amb subelements nom i adreca perquè són dues dades diferents de la mateixa entitat. Els destinataris es representen amb elements <destinatari> repetibles, i el seu tipus (per o copia) és un atribut perquè qualifica el destinatari en lloc d'aportar contingut, així es poden afegir més destinataris o altres tipus, com la còpia oculta, sense canviar l'estructura. El cos separa salutació, missatge, tancament i signatura, tal com apareixen al correu, i la signatura es desglossa en nom i càrrec. Els adjunts es col·loquen en un bloc propi perquè un correu en pot tenir zero, un o diversos, i cadascun guarda el nom del fitxer, el tipus MIME i la mida amb la unitat com a atribut. Finalment, declar la codificació UTF-8 perquè els accents i la ç es mostrin bé, i el document es podria validar amb un XSD o DTD.

JSON

En JSON he mantingut la mateixa organització lògica, amb un objecte principal correu que conté els objectes capçalera, cos i adjunts, de manera que els dos documents són equivalents. Els destinataris i els adjunts es representen amb arrays d'objectes, que és l'estructura natural de JSON per a llistes d'elements repetits, i el tipus de destinatari passa a ser un camp més de l'objecte, perquè JSON no té atributs. Aprofit els tipus natius del format: la mida de l'adjunt és un nombre, mentre que les adreces i el text són cadenes. Les claus no duen espais ni caràcters especials. JSON és més compacte i lleuger que XML, i és el més habitual per intercanviar dades en APIs, però no admet comentaris ni validació tan estricta com XSD, tot i que es pot validar amb JSON Schema.

PREGUNTA 3

XML

He dissenyat el document amb l'element arrel <mòdul>, que conté les dades pròpies del mòdul (nom, cicle, hores setmanals, caràcter obligatori i període d'impartició) i un bloc <alumnes> amb tants elements <alumne> com alumnes hi ha matriculats, de manera que es poden afegir més alumnes sense canviar l'estructura. El codi del mòdul (codi) i el NIF de l'alumne són atributs perquè són identificadors únics de cada entitat, i la resta de dades són elements, ja que són contingut i admeten subestructura, com l'adreça, que desglosso en carrer, codi postal i població per poder filtrar-hi. El nom i els cognoms van separats per poder ordenar i cercar per cognom, les dates utilitzen el format ISO 8601 per evitar ambigüitats, i el telèfon i el codi postal es guarden com a text perquè no s'operen i així no es perd el zero inicial (07300). Les faltes d'assistència són un nombre, perquè es pot sumar o

comparar amb un límit, i la nota es guarda com a text (“apte” o “no apte”) perquè és un valor qualitatiu. Finalment, declar la codificació UTF-8 perquè es vegin bé els accents i la ç, i el document es podria validar amb un XSD o DTD, cosa que el fa vàlid a més de ben format.

JSON

En JSON he mantingut la mateixa estructura lògica, amb un objecte principal modul que conté les dades del mòdul i un array alumnes d’objectes, que és l’estructura natural per a llistes d’elements repetits. Com que JSON no té atributs, el codi del mòdul i el NIF passen a ser camps de l’objecte, i l’adreça es manté com a objecte niat. Aprofit els tipus natius del format: les hores setmanals i les faltes d’assistència són nombres, mentre que el NIF, el telèfon, el codi postal i la nota són cadenes de text. Les dates segueixen el format ISO 8601 i les claus no duen espais ni caràcters especials. Així, tots dos documents són equivalents, amb la diferència que JSON és més compacte i lleuger i és el més habitual en APIs, mentre que XML permet atributs, comentaris i validació estricta amb XSD; en JSON es pot validar amb JSON Schema.